MuSEUM 実験
計測・制御プログラム操作マニュアル

第 2.1 版

最終更新：2026 年 10 月 26 日

更新履歴

版番号	更新日	変更内容
2.0	2026 年 10 月 26 日	LaTeX 版を作成。
2.1	2026 年 10 月 26 日	表題と更新履歴の記載を修正。

目次

1 スイッチの切り替え：Switchcontrol.py 4

1.1 特性評価/実験、TM110/TM210 を切り替える 4

1.2 スイッチ用電源の CH2・CH3・CH4 の出力を全て OFF にする 4

1.3 python 起動後 4

2 VNA の設定・測定・CSV 保存：test_main.py 5

2.1 VNA 接続 → 測定条件の設定 → 全 4ch を各 10 回測定 → CSV 保存をまとめて行う . 5

3 VNA の個別操作：vna_tools.py 5

3.1 Python 起動後、VNA と接続 5

3.2 測定条件・トレース・画面・マーカーを設定する 5

3.3 全 4ch を各 10 回測定し、CSV を保存する 6

3.4 現在の Narrow S11 から、最小振幅の周波数を取得する 6

3.5 操作終了後、通信を閉じる 6

4 Piezo 調整 6

4.1 スイッチを VNA 側・使用するモードに切り替える 6

4.2 ANC350 を初期化する：piezo_move.py 6

4.3 Linux PC で受信プログラムを起動する：frequency_controller.py 7

4.4 別タブで Python を起動し、送信側を準備する 7

4.5 python 起動後 7

4.6 送信側の通信を閉じる 7

5 Piezo の個別操作 8

5.1 スイッチを VNA 側・使用するモードに切り替える 8

5.2 ANC350 を初期化する：piezo_move.py 8

5.3 Linux PC で Python を起動し、ANC350 に接続する 8

5.4 現在位置を確認する 8

5.5 指定した位置へ移動する：絶対位置指定 9

5.6 現在位置から指定した距離だけ移動する：相対移動 9

5.7 指定した step 数だけ移動する 9

5.8 操作終了後、通信を閉じる 9

6 パワーの feedback 10

6.1 パワー測定値の送信を開始する：RaS_NRP2_Operator.py 10

6.2 スイッチを AMP 側・使用するモードに切り替える 10

6.3 別タブで Python を起動し、フィードバックを準備する 10

6.4 python 起動後 10

6.5 設定の変更 11

6.6 現在の状態を確認する 11

6.7 フィードバックを停止し、SG 出力を OFF にする 12

6.8 ログのパスを確認 12

6.9 測定ログのリアルタイム確認 12

7 1port 運用 12

7.1 スイッチ操作は Switchcontrol_1port.py を使用する 12

7.2 Piezo 調整の受信側は frequency_controller_1port.py を使用する 12

1　スイッチの切り替え：Switchcontrol.py

1.1　特性評価/実験、TM110/TM210 を切り替える

```
% python Switchcontrol.py AMP TM110
```

- 引数 1 VNA or AMP
- 引数 2 TM110 or TM210

1.2　スイッチ用電源の CH2・CH3・CH4 の出力を全て OFF にする

```
% python Switchcontrol.py OFF
```

1.3　python 起動後

```
>>>import Switchcontrol

>>>Switchcontrol.set_switch("VNA", "TM110")
```

- 引数 1 VNA or AMP
- 引数 2 TM110 or TM210

または

```
>>>Switchcontrol.all_off()
```

2　VNA の設定・測定・CSV 保存：test_main.py

2.1　VNA 接続 → 測定条件の設定 → 全 4ch を各 10 回測定 → CSV 保存をまとめて行う

```
% python test_main.py
```

- 保存ファイル名の末尾を指定する

```
% python test_main.py TM110_check
```

引数：ファイル名の末尾に付ける文字列

3　VNA の個別操作：vna_tools.py

3.1　Python 起動後、VNA と接続

```
>>> import pyvisa
>>> import vna_tools
>>> rm = pyvisa.ResourceManager('@py')
>>> znb = vna_tools.get_vna_resource(rm, 'TCPIP0::192.168.12.4::inst0::INSTR')
```

3.2　測定条件・トレース・画面・マーカーを設定する

```
>>> vna_tools.setup_vna(znb)
```

- TM110/TM210 それぞれの Wide/Narrow を全 4ch に設定する。

3.3 全 4ch を各 10 回測定し、CSV を保存する

```
>>> csv_path = vna_tools.measure_and_save(znb)
```

3.4 現在の Narrow S11 から、最小振幅の周波数を取得する

```
>>> f0, amp_min = vna_tools.find_min_freq(znb, 2, "Trc4")
```

- TM110：2, ”Trc4”
- TM210：4, ”Trc9”

3.5 操作終了後、通信を閉じる

```
>>> znb.close()

>>> rm.close()
```

4 Piezo 調整

4.1 スイッチを VNA 側・使用するモードに切り替える

```
% python Switchcontrol.py VNA TM110
```

- TM210 の場合は TM210 を指定する。

4.2 ANC350 を初期化する：piezo_move.py

- ANC350 を USB で Windows PC に接続する。

- Windows PC で piezo_move.py を実行し、初期化する。
- USB を Windows PC から Linux PC へ差し替える。

4.3 Linux PC で受信プログラムを起動する：frequency_controller.py

```
% python frequency_controller.py
```

- Waiting for frequency command... と表示されたら、別タブから指示を送る。
- このターミナルは起動したままにする。
- この受信側のターミナルに、現在の周波数・目標との差・移動 step 数・最終結果が表示される。
- 調整終了後は、次の指示を待ち受ける。

4.4 別タブで Python を起動し、送信側を準備する

4.5 python 起動後

```
>>> from frequency_command import FrequencyCommandProducer

>>> producer = FrequencyCommandProducer()

>>> producer.send_target("TM110", 1897400)
```

- 引数 1：TM110 or TM210
- 引数 2：目標周波数［kHz］

4.6 送信側の通信を閉じる

```
>>> producer.close()
```

5 Piezo の個別操作

5.1 スイッチを VNA 側・使用するモードに切り替える

```
% python Switchcontrol.py VNA TM110
```

- TM210 の場合は TM210 を指定する。

5.2 ANC350 を初期化する：piezo_move.py

- ANC350 を USB で Windows PC に接続する。
- Windows PC で piezo_move.py を実行し、初期化する。
- USB を Windows PC から Linux PC へ差し替える。

5.3 Linux PC で Python を起動し、ANC350 に接続する

```
>>> from pylablib.devices import Attocube

>>> atc = Attocube.ANC350()
```

- frequency_controller.py との同時接続はできない

TM110：axis1、TM210：axis2

5.4 現在位置を確認する

```
>>> atc.get_position(axis)
```

- 指定した軸の位置を表示する。単位は m。
- »> atc.get_position() で全軸の確認

5.5 指定した位置へ移動する：絶対位置指定

```
>>> atc.move_to(axis, 0.001)
```

- 例：位置 0.001 m へ移動する。
- 移動量ではなく、移動先の座標を指定する。

5.6 現在位置から指定した距離だけ移動する：相対移動

```
>>> atc.move_by(axis, 10e-6)
```

- 例：現在位置から ＋ 10 µm 移動する。
- 負方向なら-10e-6 を指定する。

5.7 指定した step 数だけ移動する

```
>>> atc.move_by_steps(axis, 10, 0.01)
```

- 引数 1：軸番号
- 引数 2：移動 step 数。負方向なら-10 などを指定する。
- 引数 3：step 間の待ち時間［秒］

5.8 操作終了後、通信を閉じる

```
>>> atc.close()
```

6　パワーの feedback

6.1　パワー測定値の送信を開始する：RaS_NRP2_Operator.py

```
% python RaS_NRP2_Operator.py
```

- このターミナルは起動したままにする。

6.2　スイッチを AMP 側・使用するモードに切り替える

```
% python Switchcontrol.py AMP TM110
```

- TM210 の場合は TM210 を指定する。

6.3　別タブで Python を起動し、フィードバックを準備する

6.4　python 起動後

```
>>> from power_feedback import PowerFeedbackController

>>> feedback = PowerFeedbackController()

>>> feedback.start(frequency_ghz=1.8974, target_power_w=7e-6,
↪  initial_amp_power_dbm=-25.0)
```

- frequency_ghz：AMP の周波数［GHz］
- target_power_w：NRP2 CH2 で測定する目標パワー［W］
- initial_amp_power_dbm：開始時の AMP 出力［dBm］
- 周波数設定 → 初期 AMP 出力設定 → AMP 出力 ON → フィードバック開始まで行う。

6.5 設定の変更

変更後の設定は、次の制御計算から使われる

目標パワーを変更する

```
>>> feedback.set_target_power(7e-6)
```

- 引数：目標パワー［W］

PID の係数を変更する

```
>>> feedback.set_pid(kp=1.0, ki=0.0, kd=0.0)
```

制御の更新間隔を変更する

```
>>> feedback.set_interval(1.0)
```

- 引数：更新ごとの待ち時間［秒］

6.6 現在の状態を確認する

```
>>> feedback.get_status()
```

- AMP 出力 ON/OFF、フィードバック状態、周波数、AMP 出力、目標値、最新の測定値などを表示する。

6.7 フィードバックを停止し、SG 出力を OFF にする

```
>>> feedback.stop()
```

6.8 ログのパスを確認

```
>>> print(feedback.log_path)
```

6.9 測定ログのリアルタイム確認

別のタブで

```
% tail -f ログのパス
```

7 1port 運用

TM110・TM210 の両方を、TM110 側の入力ポートで測定する。

7.1 スイッチ操作は Switchcontrol_1port.py を使用する

- TM110 ／ TM210 のどちらを指定しても、物理入力は TM110 側に固定される。

7.2 Piezo 調整の受信側は frequency_controller_1port.py を使用する

- 周波数調整の処理・piezo 軸・周波数範囲は従来版と同じ
- 保存する CSV・PNG の名前に 1port_input_TM110 を追加。
- 保存する図に、入力が TM110 側であることを表示。